

Project Report: Transaction User Insights

This is my own narrative write-up of the whole project, from the first look at the raw CSV to the final PDF report. It covers what I found, the decisions I made and why, and what each notebook and pipeline stage is for. The technical setup instructions live in `README.md` ; this document is the story behind the code.

1. Starting point

I was given one file, `data/raw/transactions.csv` : 278,932 rows, 17 columns, covering 1 March to 31 May 2024. My objective was twofold: engineer meaningful per-user features from this transaction history, and segment users into distinct behavioral groups. Before touching any of that, I spent time just reading the raw data closely, because every downstream decision (what a feature means, what a segment represents) depends on trusting what the data actually contains, not what I assumed it would contain.

I built the project as a staged pipeline (`main.py --stage clean|features|segment|report`) rather than a single script, specifically so each stage's output could be inspected on its own before moving to the next, and so a change deep in feature engineering would not force me to re-run geo-enrichment from scratch every time.

2. Data cleaning

2.1 The audit came before the fix

My first real decision in `src/cleaning/audit.py` was to read the CSV as raw strings with `keep_default_na=False` , specifically so hidden missing tokens (`" "` , `"N/A"` , `"null"`) would stay visible instead of pandas silently converting them to `NaN` before I could count them. That single choice is what surfaced the project's most consequential finding: 93,584 transactions (33.6%) have a blank `transaction_status` , along with six other fields (`loyalty_program` , `discount_applied` , `promo_amount` , `transaction_notes` , `merchant_rating` , `is_refunded`) blank on exactly the same rows.

I did not treat that as an error to patch over. I checked whether it correlated with time (it does not; the blank rate is flat around 33 percent every day of the three-month window), payment method, or MCC category (no concentration anywhere), and whether it changed the typical transaction value (it does not). That evidence pointed to a genuine third transaction state, not a data defect, so in `src/cleaning/standardize.py` I modeled `transaction_status_clean` as three values: `completed` , `failed` , and `pending` , with `is_completed` / `is_pending` / `is_failed` booleans derived from it. Every later module (consistency checks, feature engineering, segmentation, reporting) reads from that one column rather than re-deriving its own notion of "valid" transaction.

2.2 Flag, don't delete

The principle I held to throughout `src/cleaning/` is that original columns are never overwritten or dropped. Every inconsistency becomes a new boolean column instead: `dq_*` for genuine data quality issues (duplicate transaction IDs, non-positive amounts, promo exceeding the transaction amount, ratings on non-completed transactions, refunds on non-completed transactions, discounts with no promo amount) and `info_*` for notes that are not errors (a non-standard MCC code, a legacy OS version). `dq_any_issue`

combines all the error flags into one column for quick filtering, but nothing is ever silently corrected or removed. `merchant_rating_clean` is a good example of the pattern: it nulls out ratings on non-completed transactions for analyses that need genuine ratings, while the original `merchant_rating` column stays untouched, so the inconsistency itself is always auditable.

2.3 Geo enrichment

`geo_location` only gave me raw latitude/longitude pairs. To turn that into something usable (province, city, region), I used geopandas with the GADM administrative boundary file for Indonesia, cached locally so the pipeline does not need network access on later runs. The matching logic in `src/cleaning/geo_enrichment.py` tries point-in-polygon first (`within`), falls back to nearest-polygon-within-a-distance-limit for points that land just outside a boundary (`nearest`), and only drops to hardcoded city centroids (`centroid_fallback`) if geopandas or the boundary file is unavailable at all. Every transaction records which method matched it in `geo_match_method`, so the quality of the geo enrichment is never hidden. On this dataset, 100% of points matched `within` a polygon across 9 regencies/cities in 6 provinces, including confirming that the points one might assume are "Yogyakarta" actually fall inside Bantul Regency specifically.

2.4 Device parsing

`user_agent` only had 68 distinct values across 278,932 rows, so I parsed the unique values once (`src/cleaning/device_parser.py`) and merged the result back, rather than running a regex over every row. That gave me `os_family`, `device_type`, `os_major`, `os_version`, and an `info_legacy_os` flag for Android versions below a configurable threshold.

3. Feature engineering

`src/features/pipeline.py` builds the per-user feature table in four stages, ending at 98 columns per user (97 features plus `user_id`):

1. **Transaction-level derivation** (`transaction_level.py`): MCC group, net amount (`transaction_amount - promo_amount`), weekend/payday/Eid al-Fitr flags.
2. **Per-user aggregation** (`user_level.py`): roughly 60 columns across activity, value, category diversity, geography, payment and promo behavior, quality, time patterns, and device, each with a stated assumption (for example, Shannon entropy over a user's MCC distribution to measure variety-seeking independent of their transaction count, or the coefficient of variation of net amount to capture spend consistency, not just spend level).
3. **Big Five personality proxies** (`personality.py`): 10 columns, two per trait (a raw z-score and a 0-100 percentile). Section 3.1 below goes into this one in more depth, since it is the one feature group whose indicators I chose almost entirely on my own judgment, rather than a direct column transformation, and that deserves more scrutiny than a single summary line.
4. **Demographic proxies** (`demographics.py`): 15 columns covering 14 requested attributes (Big Five counts as one). This stage went through a real revision during the project. My first pass only attempted demographic signals that had an unambiguous MCC match (student status, parental status, vehicle ownership, homeownership, health care use), and documented everything else (gender, age, education, marital status, work location, working status, industry) as infeasible. After re-reading the task requirements, I decided that stance was too conservative: the instructions explicitly reward creative, data-driven attempts even when a signal is weak, provided the assumption is stated clearly. So I went back and built a heuristic proxy for every attribute except Industry of Employment: - **Age**: reuses the

student/parental MCC signals rather than inventing a new one, to produce an 18-24 (student-leaning) or 25-45 (parent-leaning) bracket. - **Educational Background**: presence of university-category (MCC 8220) spend, read as "currently studying," since attained education level has no signal in this data at all. - **Gender**: a composite z-score of spend share in a handful of stereotype-associated retail categories (beauty and clothing versus fuel, sporting goods, and hardware). I want to be direct about this one: it is a weak societal stereotype, not a validated or ethical ground truth, so I named the column `gender_lean_signal`, capped confidence at mostly "none" or "low," and kept it as an exploratory lean rather than a claimed identity. - **Home Location**: a user's most frequent overall transaction city. - **Work Location**: a user's most frequent city on weekdays only, when it differs from the overall dominant city. There is no time-of-day field in this dataset (only a transaction date), so I could not use the usual 9-to-5 clustering approach; a weekday/weekend split was the closest substitute the data actually supports. - **Working Status**: payday-window spending share above a threshold combined with weekend share below a threshold, as a loose proxy for regular salaried income. - **Marital/Parental Status**: a composite of the parental signal, loyalty membership, and a geographically settled primary city. I had to adjust this one after testing it: my first version required a user to transact in only one city at all, and that left almost no one eligible, because geo activity in this dataset turned out to be scattered across cities for 99% of users (the median user's single most common city only accounts for half their transactions). I replaced the strict single-city rule with a `primary_city_share >= 0.5` threshold, which produced a much more reasonable (if still small) eligible population. - **Industry of Employment**: I did attempt this one before rejecting it. I checked whether a user's own spending-category concentration (for example, heavy fuel and travel spend suggesting a delivery job) could proxy their employer's industry, and rejected the idea because it conflates what a user buys with what their employer does, two unrelated things. No field in this dataset ties a user to an employer at all, so this is the one attribute documented as infeasible in `data/processed/demographic_feasibility.csv`, with the reasoning kept rather than deleted.

Every proxy ships with its own confidence column, and most of them sit at "low" or "none" confidence for the majority of users by design: the signals genuinely are weak, and hiding that behind a false sense of precision would be worse than reporting a low coverage number honestly. For reference, coverage (share of users with a non-"none" confidence) on this dataset ranges from under 1% (marital/family proxy) to 100% (home location, which has a signal for every user almost by construction).

3.1 A closer look at the Big Five proxies

The task only pointed me to the Wikipedia page for the Big Five model itself as a reference, not to any specific method for turning transactions into a score, so the indicator-to-trait mapping in `personality.py` is something I had to design myself. I want to lay out exactly how I did that, what it is and is not based on, and why I do not treat it as more trustworthy than it actually is.

The five traits and their indicators. Each trait score is the mean of 3 to 4 existing behavioral features, each with a sign (positive or negative direction):

Trait	Indicators (sign)
Openness	category_entropy (+), n_categories (+), merchant_diversity (+)
Conscientiousness	burstiness (-), failure_rate (-), essential_share (+), payday_share (-)

Trait	Indicators (sign)
Extraversion	social_share (+), weekend_share (+), n_cities (+)
Agreeableness	avg_rating (+), merchant_repeat_ratio (+), notes_share (-)
Neuroticism	refund_rate (+), rating_std (+), cv_net_amount (+)

My reasoning when picking these was ordinary intuition about consumer behavior: someone who buys across many categories seems more open to new experience; someone whose spending is regular and rarely fails seems more conscientious; someone who spends more at social and dining merchants and across more cities seems more extraverted; and so on. That reasoning is plausible, but it is mine, not a cited methodology, and I think it would be dishonest to present it as more than that.

Where this idea actually comes from. The general premise, that personality can be inferred from spending or other behavioral records at all, is a real and active research area, not something I invented. I looked specifically for work that uses transaction data rather than social media data, and found two studies worth naming directly:

- Gladstone, Matz, and Lemaire (2019), "Can Psychological Traits Be Inferred From Spending? Evidence From Transaction Data," *Psychological Science*. This is the closest precedent to what I built: real UK bank transaction records (2,193 people, over 2 million transactions) linked to a short self-report Big Five questionnaire (the BFI-10). Their published supplemental table lists the five spending categories most correlated with each trait, and one of them lines up with a choice I made independently: they found Extraversion most positively correlated with "dining and drinking" spend, which is exactly the kind of signal my `social_share` indicator (spend share at social and dining merchants) is meant to capture for the same trait. That is a genuine point of agreement, and the only one; the rest of their top categories (flights for Openness, savings for Conscientiousness, donations for Agreeableness) rely on spending categories this dataset's 21 MCC codes do not have at all, so there was nothing to align or misalign for those four.
- Ramon, Matz, Farrokhnia, and Martens (2021), "Explainable AI for Psychological Profiling from Digital Footprints," a case study on a separate dataset of 6,408 people, also linked to a real self-report questionnaire (the BFI-2-S). Their most useful finding for me was not a specific indicator, but a pattern: Conscientiousness and Neuroticism were the most predictable traits from spending data, while Openness and Agreeableness were the least predictable. On my own dataset, Neuroticism is one of the two traits that passes its internal-consistency check, which is at least directionally consistent with that pattern, though I want to be careful not to overstate this: their result is about predicting a real external label, and mine is only about whether my own indicators agree with each other.

Self-report, not third-party annotation. Both of those studies validated their models against a real questionnaire that the actual account holder filled in about themselves, not against an outside rater's impression. That distinction matters for how this project would need to be validated if it were ever taken further: a video-based personality dataset typically has to rely on third-party annotation (crowd raters watching a clip and guessing a score) because there is no practical way to ask a random public-video subject to complete a questionnaire. Transaction data does not have that constraint. If this were a real product with real users, the correct path is a short self-report instrument, like the 10-item BFI-10 or the even shorter TIPI, collected once during onboarding with consent, and then compared against these behavioral indicators the same way Gladstone and Ramon's teams compared theirs. There is no equivalent path through third-party annotation here, since nobody looking at a transaction history could meaningfully guess

someone's personality by eye the way a video gives them something to actually observe.

What Cronbach's alpha does and does not tell me. I added an alpha check per trait because I did not want to publish five scores that all look equally credible when I know they are not. It is important to be precise about what that check actually measures: alpha only tells me whether the 3 to 4 indicators behind one trait move together across users in this dataset. It says nothing about whether the resulting score matches a real person's real personality, because no real personality label exists here to check against. On this run, Openness (alpha 0.99) and Neuroticism (alpha 0.76) pass a reasonable internal consistency bar; Conscientiousness (-0.44), Extraversion (0.41), and Agreeableness (-0.04) do not, meaning their own indicators barely correlate with each other, let alone with anything external. I report all five in the PDF regardless, but I flag the three that fail as not interpretable as personality, rather than quietly dropping them or, worse, presenting every trait as equally reliable because the average looks fine.

In short: the concept is grounded in real published work, one specific indicator happens to agree with that work's own finding, and the validation method I added (Cronbach's alpha) is honest about what it can and cannot tell me. What it cannot do, and what no amount of internal-consistency checking can substitute for, is prove these scores correspond to real personality, because this dataset has no real people behind its user IDs to check against in the first place.

4. Segmentation

`src/segmentation/` runs K-Means on 15 behavioral features only: recency, frequency, value, category and merchant diversity, discount and loyalty usage, payment mix, and quality. Geography, device, payment-method category, personality scores, and the demographic proxies from section 3 are all deliberately excluded from the clustering input. I made that choice so a segment would reflect actual behavior rather than being a relabeled version of a column I already had, and so those excluded fields could instead be used afterward, purely to describe segments once they already existed.

I chose the number of clusters automatically rather than picking one that looked good: the pipeline fits K-Means for $k = 3$ through 6 and scores each with silhouette, selecting the best one. On this dataset that was $k = 5$, with a silhouette score of 0.37, which is modest rather than textbook-clean. I reported that plainly instead of searching for a k that looked more impressive, because the modest score is consistent with the same weak correlation structure already visible in the Big Five Cronbach's alpha results.

This segmentation goes beyond a traditional RFM split on purpose. Recency, frequency, and monetary value are in there (`recency_days`, `n_valid_purchases`, `avg_net_amount`), but so are diversity (`category_entropy`, `merchant_diversity`), promo and loyalty behavior (`discount_tx_share`, `loyalty_member`), payment mix (`credit_card_share`), quality (`failure_rate`, `refund_rate`), and time patterns (`weekend_share`, `lebaran_lift`), so segments can separate on behavior that RFM alone would never capture.

The five segments that came out, and a few things I found interesting about each:

- **Bargain Hunters** (45.8% of users): the largest segment, with the "currently studying" and 18-24 age-bracket signals both appearing at 1.6 times the overall rate.
- **New / Low Engagement** (32.4%): I found this one is almost entirely made up of a specific population, users with essentially zero completed purchases, whose transactions are all pending or failed. I confirmed that overlap directly in notebook 02 (see below): this cluster is close to 100% "all-pending" users when checked against a raw status count computed completely independently of the clustering.

- **Regular / Steady Spenders** (14.7%): stands out at 6.3 times the overall rate on the family/parental signal, and this segment also turns out to contain 92% of all loyalty program members, a connection the clustering found on its own without loyalty membership ever being fed into it.
- **At-Risk / Dormant** (5.6%): the only segment with no demographic proxy notably over-represented. Longer recency (25 days since the last transaction versus 7 for most other segments) and the lowest activity are what define it instead.
- **Champions (High-Value Loyal)** (1.5%): the smallest segment, with the family/parental and male-leaning retail signals both at 1.7 times the overall rate.

For the segment stories in the report, I deliberately avoided reporting each segment's raw top demographic category, because for most of these proxies the raw top category is just the majority "no signal" or "unknown" value in every segment (the signals are too rare to dominate any segment's top-3 breakdown on their own). Instead I compute how over- or under-represented a rare signal is in a given segment relative to the whole user base, and only mention a trait in the story if it is notably over-represented (at least 1.3 times the overall rate). That is what produced the specific, differentiated findings listed above, rather than five segments that all read the same way.

5. Reporting

`src/reporting/pipeline.py` turns all of the above into a 21-page PDF (`outputs/reports/transaction_insights_report.pdf`), built from live numbers computed on each run, not static text. The chapter order was itself a decision I revisited partway through the project: the core deliverables, feature methodology and segmentation, now come first (chapters 3 and 4), with a short data audit before them for context (chapter 2) and assumptions/limitations after them (chapter 5). Everything beyond that, the regional and time-pattern charts, and eight business questions I explored for additional insight (repeat purchase by region, merchant-level discount patterns, category performance, device and OS coverage, balance versus credit card, rating drivers, loyalty engagement, and regional promo usage), is grouped into a clearly labeled supplementary appendix (chapter 6), so a reader can tell at a glance what answers the core requirements and what goes beyond them. Every statistical test in that appendix reports an effect size alongside the p-value (Cramer's V for chi-square tests, for example), and every per-segment breakdown includes the sample size `n`, specifically so a result computed from a handful of rows (like most metrics for the New / Low Engagement segment, which by definition has almost no completed purchases) is never mistaken for one computed from thousands.

6. The notebooks

I treat the three notebooks in `notebooks/` as the evidence trail behind the production code, not a separate exercise written after the fact. Each one calls the real project code directly (via `inspect.getsource` for the first two, and via direct imports from `src/segmentation/` for the third) rather than re-implementing anything, so re-running a notebook after a code change shows the current behavior, not a frozen snapshot.

6.1 `01_edu.ipynb`, exploratory data audit

This is where the project starts. It validates the assumptions I had written down before looking closely at the real file (row counts, user counts, specific inconsistency counts) against the actual CSV, and traces each confirmed or contradicted finding into the specific file that decision shaped: `src/cleaning/consistency.py` for the rating and refund inconsistency checks, `src/features/transaction_level.py` and `config.yaml` for the Eid al-Fitr volume lift, and `src/cleaning/geo_enrichment.py` for the 100% geo-match result. The single largest finding it surfaces,

the blank transaction status, gets flagged here but is handed off to its own dedicated notebook rather than being investigated in a few cells, because it ended up being consequential enough to deserve that.

6.2 02_blank_status_investigation.ipynb , deep dive on the blank status

This notebook exists because one finding from notebook 01 was too important to leave as a quick aside. It reproduces the 33.6% blank-status count from scratch, then spends most of its length ruling out alternative explanations before trusting that it is a genuine third transaction state: a check against the date (no time-cutoff pattern), against payment method and MCC (no concentration), against transaction value (no difference), and against individual users (about a third of users turn out to have zero resolved transactions at all in this three-month window). Only after that evidence is laid out does it walk through the decision to model this as a `pending` status rather than dropping the rows or reclassifying them as failed, and then traces that decision live into every file it touches: `standardize.py` , `consistency.py` , `transaction_level.py` , the data quality report, and the report's limitations text. It closes by confirming, using the actual pipeline outputs, that the "New / Low Engagement" segment found independently in notebook 03 is almost entirely made up of the same "all-pending" users identified here through a completely different method, which is about as strong a cross-check as two independent analyses can give each other.

6.3 03_user_segmentation.ipynb , feature selection and clustering

This notebook is where I worked out and justified every modeling decision behind `src/segmentation/` : why 15 of the roughly 70 candidate features were chosen for clustering and the rest excluded (redundant monetary columns, compositional category shares that would otherwise dominate purely by column count, and anything already the subject of its own proxy or business question), how the `inf` values in `lebaran_lift` get cleaned before either scaling or reporting, the elbow and silhouette plots behind the `k = 5` choice, a PCA projection of the clusters for a visual sanity check, and the rule that turns raw cluster centroids into the five named personas. Two real issues surfaced and got fixed while building this notebook, not after: an `inf` value leaking into the raw (unscaled) segment means reported in `segment_profiles.json` , and the risk of reading a rate computed from a single-digit sample size (a handful of "New / Low Engagement" users with any completed purchase at all) as if it carried the same weight as one computed from thousands. Both are now handled directly in the production code (`clean_raw_features` and the `n`-reporting helper in `src/reporting/analyses.py`), not just noted here.

7. Testing and reproducibility

The whole pipeline runs from a clean checkout with `uv sync` followed by `python main.py` , and reproduces the exact same outputs because every parameter (missing-value tokens, category values, geo settings, calendar windows, segmentation feature list, significance level) lives in `config.yaml` , not hardcoded in the code. 26 pytest tests cover the cleaning flags, geo-matching logic (including a synthetic boundary file so the tests do not need network access), feature statistics (entropy, burstiness, Cronbach's alpha), every new demographic proxy, the segmentation functions, and a full end-to-end smoke test that runs `main.py` against a small synthetic dataset and checks that a PDF comes out the other end. If geopandas or pyarrow are not available in an environment, the pipeline degrades gracefully (centroid-based geo fallback, CSV.gz instead of Parquet) rather than failing outright.

8. What I would still want before trusting this in production

I want to be direct about the limits of this work, the same way the report itself is. The dataset is synthetic, and that shows up in a few places: most of the statistical tests in the report's appendix are not significant, three of the five Big Five proxy traits fail their own internal-consistency check, and most demographic proxies sit at low or no confidence for the majority of users. None of the demographic or personality proxies here have been validated against real self-reported ground truth (an actual short personality survey, like a BFI-10 or TIPI, linked back to real users), which is the only way I would trust them enough to use them for anything beyond exploratory segmentation. There is also no promo-source field, so the merchant-funded-versus-platform-funded discount split in the appendix is a statistical heuristic, not a verified label. If this were handed a real dataset next, those are the three gaps I would close first.